CIPHERMESH

MULTI-AGENT COORDINATION

Protocol execution, utility-driven negotiation, and consensus handshake

DELIVERABLE Complete runnable source package + reproducible terminal trace

STUDENT Kumar Waibhav Akshat

REGISTRATION ID RA2212704010027

PROGRAMME Integrated M.Tech - Computer Science & Engineering (Data Science)

SECTION AG1

VERSION / DATE 0.3.0 | 12 October 2026

ENGINEERING OUTCOME

Two independent agents negotiate to a price both accept, then commit the same contract only after matching READY and COMMIT_ACK messages.

01 Executive summary

CipherMesh v0.3.0 adds a controlled multi-agent negotiation module alongside the existing LLM privacy policy and TCP messaging components. The demo instantiates a workload buyer and an inspection-capacity supplier as distinct agent objects. Each computes its own utility, evaluates offers, and adjusts its stance through a bounded concession rule.

The negotiation follows a Contract-Net-inspired sequence: CALL_FOR_PROPOSAL → PROPOSAL → alternating COUNTEROFFER messages → ACCEPT → PREPARE/READY → COMMIT/COMMIT_ACK. An accepted price is only a candidate until both participants confirm the same SHA-256 contract digest during the consensus handshake.

5	$143.28	20
offer rounds	per 1,000 requests	serialized messages

Engineering acceptance criteria

Criterion	How this implementation addresses it
Multi-instance stability	Unique buyer/supplier IDs, bounded 1-100 turn limit, deterministic inputs, strict typed envelopes.
Negotiation behavior	CFP, supplier proposal, alternating counteroffers, explicit accept/reject and turn-based concession.
Utility-based stance	Role-specific utility formulas, reservation utility and decaying aspiration threshold.
Consensus termination	Both agents validate the same contract digest, stage READY, commit, and acknowledge COMMIT_ACK.
Traceability	Every request/response is JSON serialized and revalidated; terminal events expose stage, route and payload summary.
Failure behavior	Low budget, unmet service levels, invalid schemas and transport-size violations do not produce a false agreement.

Scope statement

The negotiation bus is an in-process JSON transport used for deterministic reproducibility. It is not a remote TCP negotiation endpoint. The project separately retains a TCP client/server demonstration for networked state synchronization. Capacity, latency and recall values in this negotiation scenario are synthetic configured inputs, not measured benchmarks.

02 Scenario and architecture

The scenario models an application workload that needs privacy-inspection capacity before LLM requests are forwarded. The buyer prioritizes price while enforcing capacity, p95 latency and recall constraints. The supplier protects a minimum acceptable margin and gradually concedes from its opening ask.

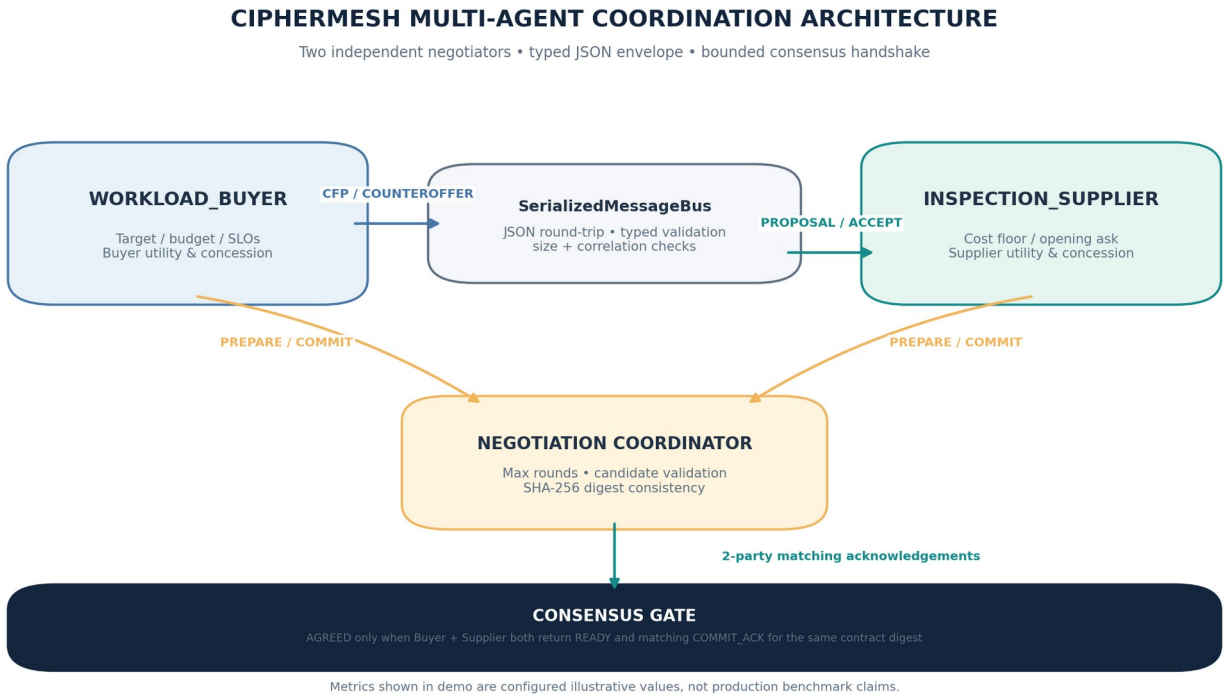


Figure 1. Two independent participants, serialized message bus, and coordinator-owned consensus gate.

Responsibility boundaries

Component	Responsibilities	Does not do
WORKLOAD_BUYER	Initiates CFP; checks service constraints; computes buyer utility; sends price counteroffers.	Does not change the supplier utility formula or negotiate above its maximum budget.
INSPECTION_SUPPLIER	Returns proposal; computes margin utility; makes bounded ask concessions; accepts only above reservation utility.	Does not override buyer service-level constraints.
SerializedMessageBus	JSON encode/decode; Pydantic schema validation; frame-size limit; sender/recipient/correlation check; timeline.	Does not determine a fair price or bypass an agent decision.
Coordinator	Bounds the loop; validates bilateral candidate; runs PREPARE/COMMIT; classifies AGREE/FAILED/IN_DOUBT.	Does not claim success from one acceptance or one acknowledgement.

Why this split matters

Negotiation, transport validation and agreement finalization are separated. This keeps utility policy local to each agent and makes the transport replaceable later without changing the buyer/supplier logic. A future multi-host implementation can reuse the message stages but must add authenticated networking and durable recovery.

03 Protocol state machine

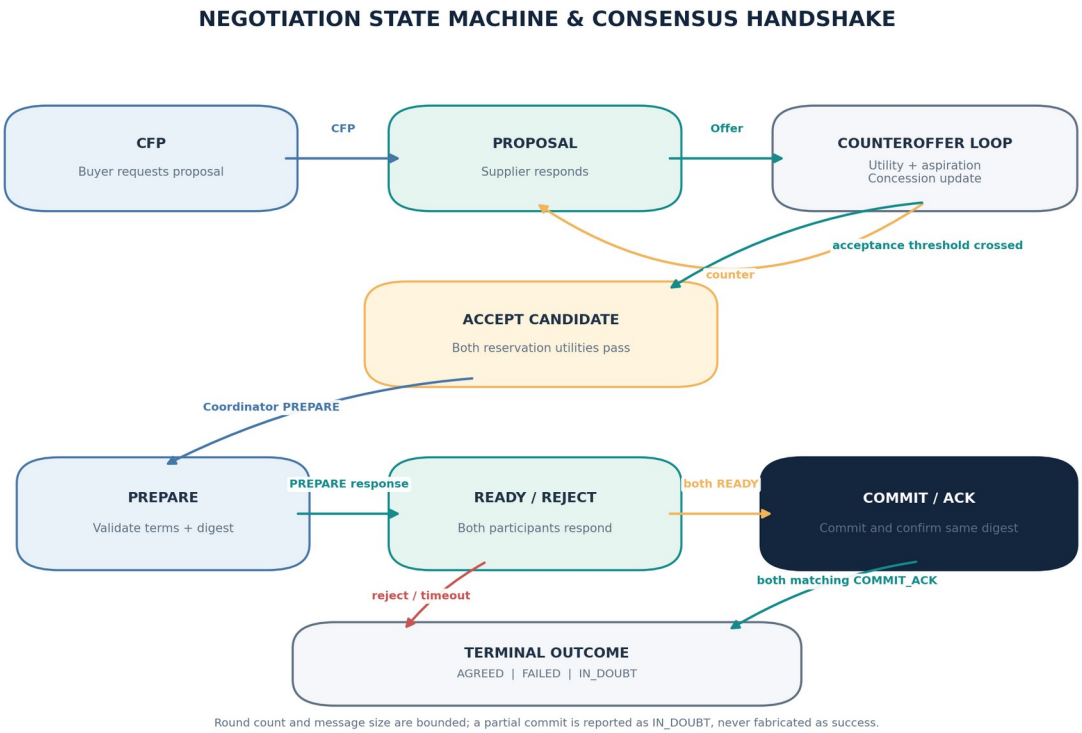


Figure 2. Offer loop, candidate validation, two-participant handshake, and explicit terminal outcomes.

Execution phases

A. CALL_FOR_PROPOSAL Buyer sends maximum budget, target price and minimum service constraints; supplier returns a typed proposal.

B. PROPOSAL / COUNTEROFFER Both agents evaluate price with independent utility functions. Each counteroffer includes utility and aspiration context; concession increases as turns advance.

C. ACCEPT / REJECT An offer must meet the receiver's current aspiration and reservation utility. The buyer also checks capacity, p95 latency and minimum recall.

D. PREPARE / READY Coordinator sends identical terms and a canonical SHA-256 digest to both participants. Each validates and stores a pending contract before replying READY.

E. COMMIT / COMMIT_ACK Each agent commits only a matching prepared digest. Coordinator reports AGREED after both distinct agents return matching acknowledgements.

Terminal outcomes

Outcome	Meaning
AGREED	Both participants committed the same digest and both COMMIT_ACK responses were received.
FAILED	No acceptable offer, service constraint failure, invalid data, or transport failure occurred before commit.
IN_DOUBT	One commit may have happened, but the coordinator cannot confirm both acknowledgements. Reconciliation is needed; consensus is not claimed.

04 Utility model and rational stance update

The model is intentionally linear and fully visible so that a student can explain every offer. Utility outputs are model values in the bounded range 0-100, not learned reward scores.

Buyer utility

```
U_B(p) = 100 × clip((p_max - p) / (p_max - p_target), 0, 1)
p_target = $112.00    p_max = $180.00    reservation utility = 40
```

At or below the buyer target price, utility is 100. It falls linearly toward 0 at the budget ceiling. Offers beyond the buyer's budget are never emitted because the counteroffer is clipped at p_max.

Supplier utility

```
U_S(p) = 100 × clip((p - c_floor) / (p_ask - c_floor), 0, 1)
c_floor = $100.00    p_ask = $176.00    reservation utility = 40
```

At the opening ask, supplier utility is 100. Utility declines toward 0 at the supplier cost floor. With a minimum utility of 40, the supplier protects the corresponding reservation price rather than accepting every buyer bid.

Dynamic aspiration and concessions

```
Aspiration(t) = max(reservation_utility, 92 - 9t)
Buyer concession αB(t)   = min(0.40, 0.12 + 0.035t)
Supplier concession αS(t) = min(0.42, 0.14 + 0.040t)
Next stance = current stance + α(t) × (other stance - current stance)
```

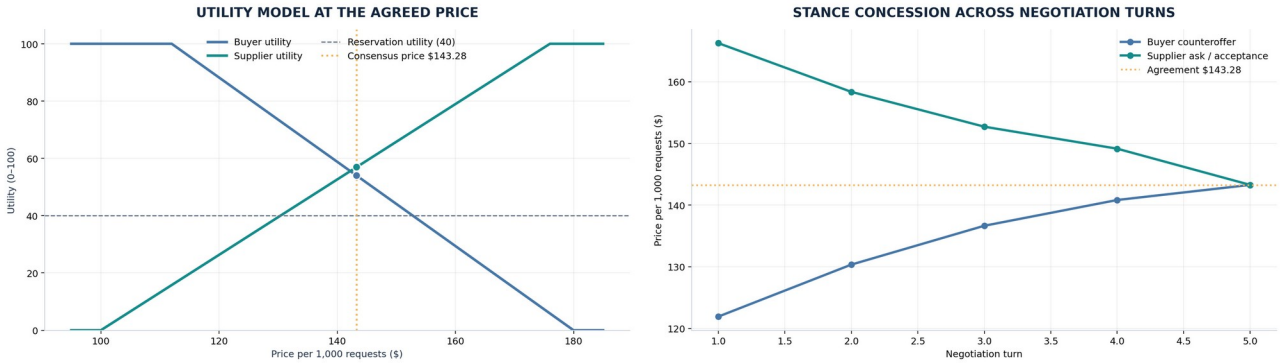


Figure 3. Left: utility curves and reservation floor. Right: price stances converging to the fifth-round agreement.

05 Turn-by-turn negotiation timeline

Run the default example with python -m ciphermesh.negotiation.demo. Each line represents a JSON-serialized, schema-validated request or response. The complete console output and a machine-readable event log are bundled under docs/examples/.

Turn	Supplier ask received	Buyer counteroffer	Buyer utility	Supplier decision
1	$176.00	$121.92	85.4	Counters at $166.27
2	$166.27	$130.35	73.0	Counters at $158.37
3	$158.37	$136.65	63.8	Counters at $152.72
4	$152.72	$140.83	57.6	Counters at $149.15
5	$149.15	$143.28	54.0	Accepts buyer counteroffer

Agreement economics at turn 5

Measure	Agreed value	Validation
Price / 1,000 requests	$143.28	Within buyer budget and supplier reservation range.
Buyer utility	54.00 / 100	Exceeds buyer reservation utility of 40.
Supplier utility	56.95 / 100	Exceeds supplier reservation utility of 40 and turn-5 aspiration of 47.
Capacity	600 req/s	Meets buyer minimum of 400 req/s.
p95 latency	18 ms	Meets buyer maximum of 30 ms.
Estimated recall	97%	Meets buyer minimum of 95%; scenario assumption only.

Execution evidence

```
Status         : AGREED
Rounds         : 5
Wire messages  : 20
Timeline events : 21 (includes one local coordinator note)
Termination    : consensus handshake completed; matching COMMIT_ACK from both agents
```

The full terminal text is saved at docs/examples/multi_agent_negotiation_timeline.txt; docs/examples/negotiation_result.json contains the contract, digest, utilities and ordered events.

06 Consensus handshake and message integrity

Acceptance is not the same as consensus. The implementation introduces an explicit commit boundary so both agents validate and store the same terms before the coordinator announces an agreed result.

Step	Request / response	Guardrail
1	Coordinator → Buyer: PREPARE	Schema validation; full contract included; digest supplied.
2	Buyer → Coordinator: READY	Buyer checks service constraints, buyer reservation utility and digest; stores pending terms.
3	Coordinator → Supplier: PREPARE	Same canonical terms and digest; independent supplier checks.
4	Supplier → Coordinator: READY	Supplier verifies reservation utility and digest; stores pending terms.
5	Coordinator → Buyer: COMMIT	Rejected unless a matching PREPARE created pending state.
6	Buyer → Coordinator: COMMIT_ACK	Returns committed digest.
7	Coordinator → Supplier: COMMIT	Same digest required.
8	Supplier → Coordinator: COMMIT_ACK	Returns committed digest. Only then consensus is reported.

Envelope and safeguards

- Schema version, fresh message ID, negotiation ID, sender, recipient, stage, turn, timestamp, correlation ID and payload are explicit fields.
- Every message is serialized to UTF-8 JSON and parsed again through the Pydantic schema; unknown envelope fields are rejected.
- Outbound and response payload sizes are bounded; replies must match the request sender/recipient and correlation ID.
- The coordinator enforces a finite round count. A low-budget scenario exits FAILED instead of looping indefinitely.
- The SHA-256 contract digest detects different term payloads; it is not a digital signature and does not authenticate an agent.

Distributed-systems caveat

If one agent commits and its acknowledgment is lost, a coordinator cannot know from silence whether that remote commit happened. This implementation reports IN_DOUBT if a commit acknowledgement is missing. A production version needs durable transaction records, idempotent retries, authenticated channels and a reconciliation workflow.

07 Code map and execution instructions

Module	What it implements
negotiation/models.py	Strict message stages, contract terms, timeline events and result statuses.
negotiation/agents.py	Buyer/supplier utility models, constraints, counteroffers, PREPARE/READY and COMMIT behavior.
negotiation/transport.py	JSON round-trip, typed validation, payload limit, routing/correlation checks, event recorder.
negotiation/protocol.py	Bounded CFP/offer loop, acceptance checks, bilateral consensus handshake and termination.
negotiation/demo.py	CLI timeline, summary, optional JSON export, deliberate failure scenario.
tests/test_negotiation.py	Success, failure, SLO rejection, digest, identity and handshake-state tests.

Install

```
python -m venv .venv
# Windows PowerShell: .\.venv\Scripts\Activate.ps1
# macOS/Linux:       source .venv/bin/activate
python -m pip install --upgrade pip
pip install -e ".[dev]"
```

Run the successful negotiation

```
python -m ciphermesh.negotiation.demo
python -m ciphermesh.negotiation.demo --json-out docs/examples/negotiation_result.json
```

Run a bounded no-agreement scenario

```
python -m ciphermesh.negotiation.demo --buyer-max-price 120 --max-rounds 6
```

The second command intentionally returns exit status 2 because it terminates without consensus. This is expected and should be used to demonstrate that the round limit works and failure is explicit.

Run the full test suite

```
python -m pytest -q
```

08 Verification, limitations and next steps

Verification performed

Test / check	Expected outcome
Default negotiation integration test	AGREED; at least three rounds; both agents store identical committed terms under the same digest.
Timeline stage assertions	CFP, proposal, counteroffer, PREPARE, READY, COMMIT, COMMIT_ACK and consensus marker appear.
Restrictive budget	FAILED within the configured round limit; no agreed contract.
Unacceptable supplier SLO	Buyer rejects the supplier profile before negotiating an agreement.
Deterministic digest	Equivalent contract objects produce the same SHA-256 digest.
Participant identity	Coordinator rejects agents with duplicate IDs.
Handshake / transport integrity	COMMIT without PREPARE and a tampered contract digest are rejected; oversized messages fail closed.
Existing CipherMesh regression suite	Privacy policy, TCP framing, network handler and gateway tests remain passing.

Observed automated test result: 31 passed.

Known limitations

- Local transport: this module is intentionally deterministic and in-process; independent operating-system processes and remote hosts are not part of this exercise.
- Synthetic profile metrics: capacity, latency and estimated recall are configured values. No model is called and no recall benchmark is calculated.
- Simplified economics: a single price is negotiated; service metrics are hard constraints rather than separate price-quality negotiation dimensions.
- In-memory state: agent pending and committed terms are not durable across process restart.
- Security: SHA-256 provides consistency, not authenticity. Remote deployment requires mTLS or signed messages, per-agent authorization, replay-safe persistence and secret management.

Recommended engineering roadmap

1. Implement a TCP or JSON-RPC transport adapter for the same typed negotiation envelope while preserving the current local deterministic test transport.
2. Persist proposal, pending, committed and idempotency states; simulate a dropped COMMIT_ACK and verify IN_DOUBT recovery/reconciliation.
3. Extend the CFP to multiple suppliers; score feasible bids and select a winner before reusing the two-party handshake.
4. Calibrate utility parameters from explicit stakeholder input and test robustness over parameter sweeps; report agreement rate, utility distribution and average turns.
5. Replace synthetic performance values with controlled benchmark measurements before making product claims.

Conclusion

This implementation demonstrates utility-driven negotiation between two distinct agents, bounded counteroffers, and a verifiable PREPARE/COMMIT handshake. It also demonstrates explicit no-consensus termination.